

Profiling and Optimizing LLM Inference at Scale

Saksham Rathi

Bachelor's Thesis Project - Stage 1
Under Prof. Mythili Vutukuru

Indian Institute of Technology Bombay
Autumn 2025-26

Contents

1. Introduction
2. Background
3. Prior Work
4. Experiments - Single Family
5. Metrics
6. Experiments - Multiple Families
7. Future Steps

Motivation

- Many users share the same prefix/system prompt
- Example 1: OS TA Chatbot with shared lecture notes
- Example 2: Enterprise assistants over shared product docs
- Recomputing attention over identical prefixes = waste
- Large batch sizes → exploding KV-cache memory usage

Motivation



- Many users share the same prefix/system prompt
- Example 1: OS TA Chatbot with shared lecture notes
- Example 2: Enterprise assistants over shared product docs
- Recomputing attention over identical prefixes = waste
- Large batch sizes → exploding KV-cache memory usage

Motivation



- Many users share the same prefix/system prompt
- Example 1: OS TA Chatbot with shared lecture notes
- Example 2: Enterprise assistants over shared product docs
- Recomputing attention over identical prefixes = waste
- Large batch sizes → exploding KV-cache memory usage

Motivation



- Many users share the same prefix/system prompt
- Example 1: OS TA Chatbot with shared lecture notes
- Example 2: Enterprise assistants over shared product docs
- Recomputing attention over identical prefixes = waste
- Large batch sizes → exploding KV-cache memory usage

Motivation



- Many users share the same prefix/system prompt
- Example 1: OS TA Chatbot with shared lecture notes
- Example 2: Enterprise assistants over shared product docs
- Recomputing attention over identical prefixes = waste
- Large batch sizes $\rightarrow$ exploding KV-cache memory usage

Problem Statement



- Characterize LLM inference performance with shared prefixes
- Identify key GPU-level performance metrics
- Examine shared-prefix optimizations in inference engines
- Study effects of prefix caching, chunked prefill, scheduling
- Understand scalability bottlenecks with many prefix families

Problem Statement



- Characterize LLM inference performance with shared prefixes
- Identify key GPU-level performance metrics
- Examine shared-prefix optimizations in inference engines
- Study effects of prefix caching, chunked prefill, scheduling
- Understand scalability bottlenecks with many prefix families

Problem Statement



- Characterize LLM inference performance with shared prefixes
- Identify key GPU-level performance metrics
- Examine shared-prefix optimizations in inference engines
- Study effects of prefix caching, chunked prefill, scheduling
- Understand scalability bottlenecks with many prefix families

Problem Statement



- Characterize LLM inference performance with shared prefixes
- Identify key GPU-level performance metrics
- Examine shared-prefix optimizations in inference engines
- Study effects of prefix caching, chunked prefill, scheduling
- Understand scalability bottlenecks with many prefix families

Problem Statement



- Characterize LLM inference performance with shared prefixes
- Identify key GPU-level performance metrics
- Examine shared-prefix optimizations in inference engines
- Study effects of prefix caching, chunked prefill, scheduling
- Understand scalability bottlenecks with many prefix families

Prefill vs Decode



- **Prefill:** Takes the whole user prompt as input and computes the probability of the first new token. Generate key and value vectors. Can be parallelized because all the tokens are known.
- **Decode:** Generates the remaining new tokens sequentially. Stops when the maximum length is reached, or when an end-of-sequence token is emitted. Autoregressive in nature.

Prefill vs Decode



- **Prefill:** Takes the whole user prompt as input and computes the probability of the first new token. Generate key and value vectors. Can be parallelized because all the tokens are known.
- **Decode:** Generates the remaining new tokens sequentially. Stops when the maximum length is reached, or when an end-of-sequence token is emitted. Autoregressive in nature.

Inference Engines

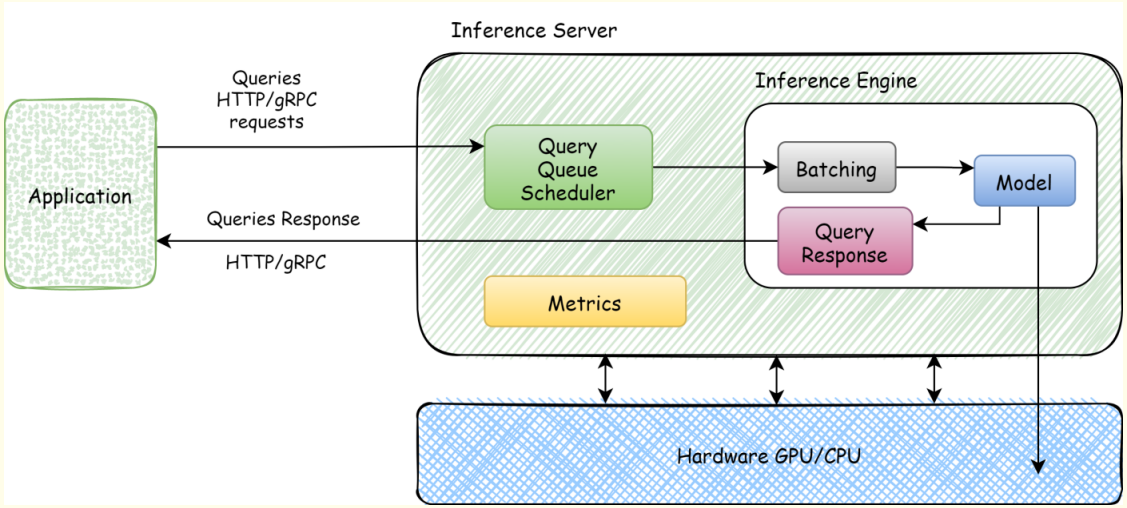


Figure: Architecture of an Inference Engine

- Systems layer for managing LLM inference on hardware
- Batch multiple requests to improve GPU utilization
- Memory: KV-cache reuse, shared buffers, quantization

Inference Engines

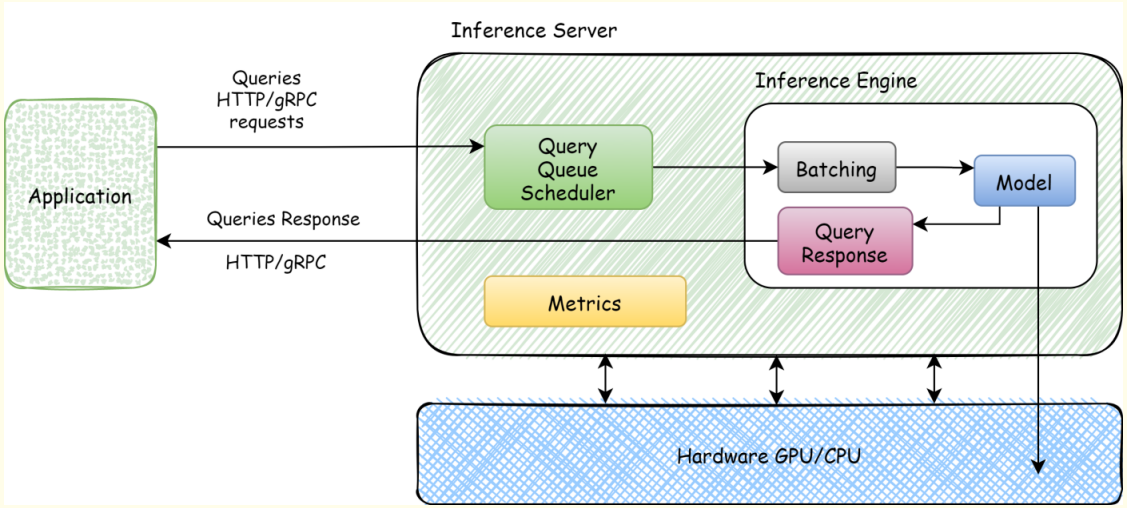


Figure: Architecture of an Inference Engine

- Systems layer for managing LLM inference on hardware
- Batch multiple requests to improve GPU utilization
- Memory: KV-cache reuse, shared buffers, quantization

Inference Engines

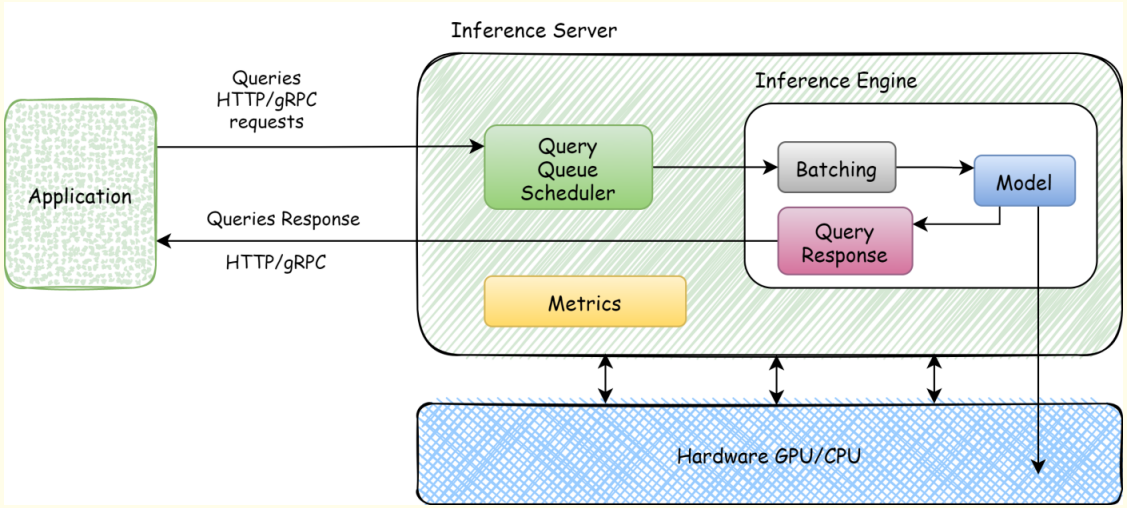


Figure: Architecture of an Inference Engine

- Systems layer for managing LLM inference on hardware
- Batch multiple requests to improve GPU utilization
- Memory: KV-cache reuse, shared buffers, quantization

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

vLLM

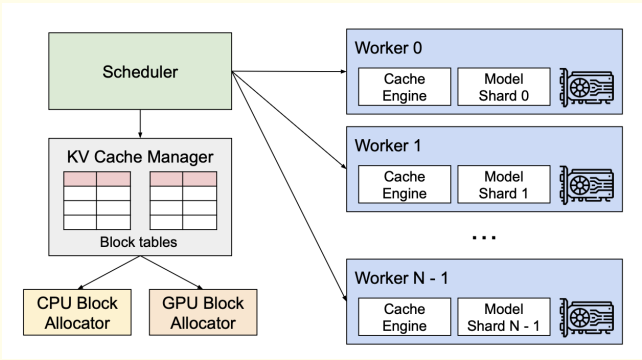


Figure: vLLM system overview

- Centralized scheduler for coordinating GPU workers
- **PagedAttention:** splits KV cache into fixed-size blocks
- Eliminates internal/external fragmentation
- Shared blocks for identical prefixes / beam search (copy-on-write)
- Evicted blocks moved to CPU memory
- Each block tracked via a hash (parent hash + tokens + metadata)

vLLM

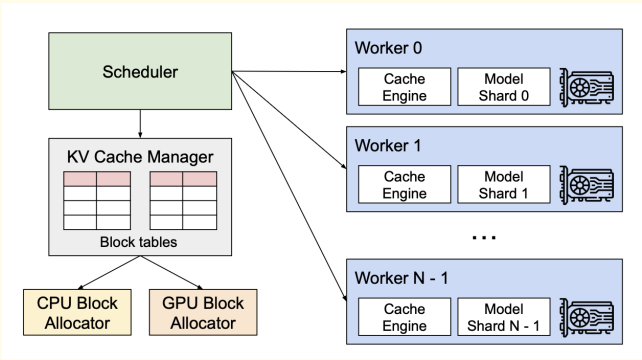


Figure: vLLM system overview

- Centralized scheduler for coordinating GPU workers
- **PagedAttention:** splits KV cache into fixed-size blocks
- Eliminates internal/external fragmentation
- Shared blocks for identical prefixes / beam search (copy-on-write)
- Evicted blocks moved to CPU memory
- Each block tracked via a hash (parent hash + tokens + metadata)

vLLM

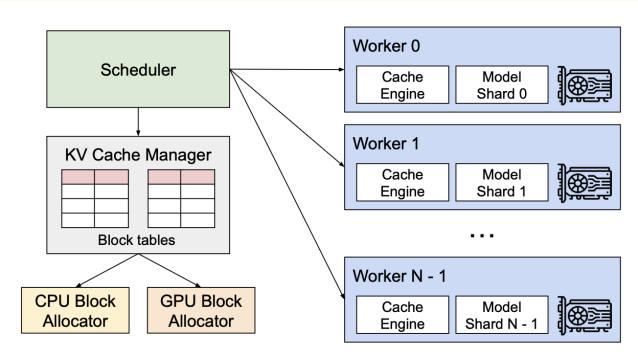


Figure: vLLM system overview

- Centralized scheduler for coordinating GPU workers
- **PagedAttention:** splits KV cache into fixed-size blocks
- Eliminates internal/external fragmentation
- Shared blocks for identical prefixes / beam search (copy-on-write)
- Evicted blocks moved to CPU memory
- Each block tracked via a hash (parent hash + tokens + metadata)

vLLM

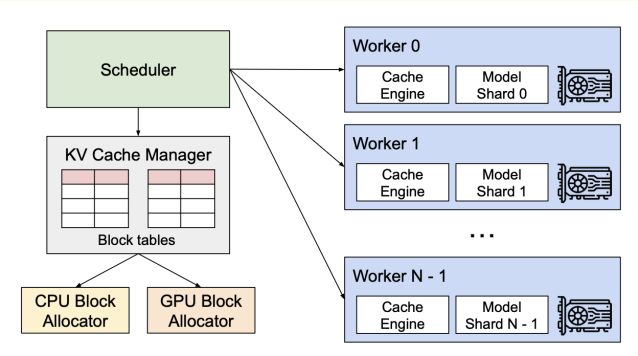


Figure: vLLM system overview

- Centralized scheduler for coordinating GPU workers
- **PagedAttention:** splits KV cache into fixed-size blocks
- Eliminates internal/external fragmentation
- Shared blocks for identical prefixes / beam search (copy-on-write)
- Evicted blocks moved to CPU memory
- Each block tracked via a hash (parent hash + tokens + metadata)

vLLM

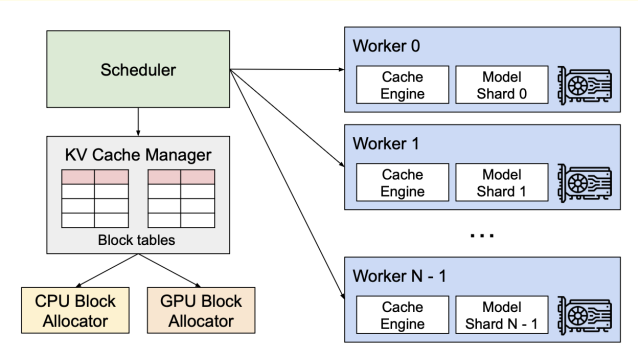


Figure: vLLM system overview

- Centralized scheduler for coordinating GPU workers
- **PagedAttention:** splits KV cache into fixed-size blocks
- Eliminates internal/external fragmentation
- Shared blocks for identical prefixes / beam search (copy-on-write)
- Evicted blocks moved to CPU memory
- Each block tracked via a hash (parent hash + tokens + metadata)

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

vLLM

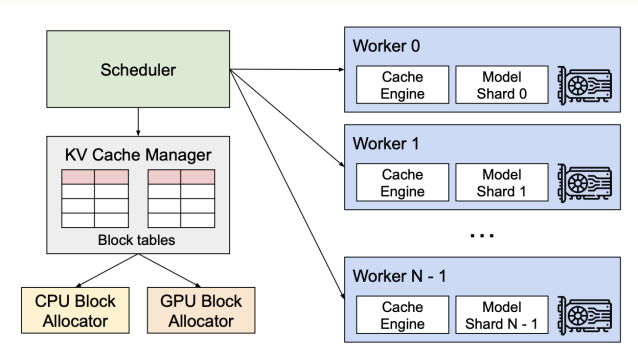


Figure: vLLM system overview

- Centralized scheduler for coordinating GPU workers
- **PagedAttention:** splits KV cache into fixed-size blocks
- Eliminates internal/external fragmentation
- Shared blocks for identical prefixes / beam search (copy-on-write)
- Evicted blocks moved to CPU memory
- Each block tracked via a hash (parent hash + tokens + metadata)

FastTree

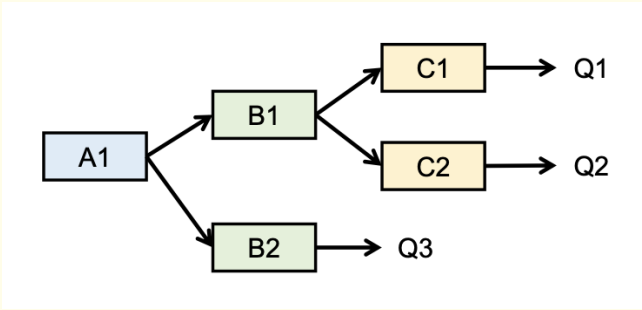


Figure: KV Cache Sharing through RadixTree

- GPU kernels operate over shared prefixes via a radix tree
- Optimal edge assignment minimizes tile padding + intermediate-state loss
- Merges same-prefix queries into one kernel → fewer memory accesses, GEMV → GEMM

FastTree

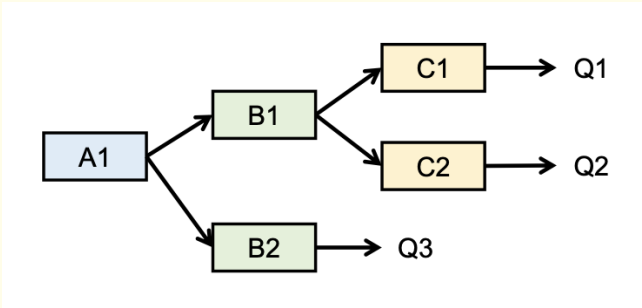


Figure: KV Cache Sharing through RadixTree

- GPU kernels operate over shared prefixes via a radix tree
- Optimal edge assignment minimizes tile padding + intermediate-state loss
- Merges same-prefix queries into one kernel → fewer memory accesses, GEMV → GEMM

FastTree

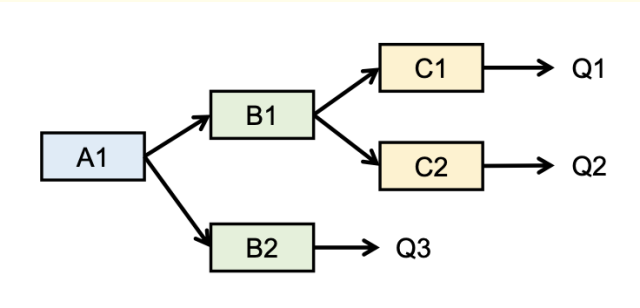


Figure: KV Cache Sharing through RadixTree

- GPU kernels operate over shared prefixes via a radix tree
- Optimal edge assignment minimizes tile padding + intermediate-state loss
- Merges same-prefix queries into one kernel $\rightarrow$ fewer memory accesses, GEMV $\rightarrow$ GEMM

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

BatchLLM



- Accelerates large-batch LLM inference
- **Global Prefix Sharing:** Build prefix tree *ahead-of-time*; avoids LRU eviction; near-optimal KV reuse
- **Group Scheduling:** Execute requests as prefix-sharing groups; compute prefix once; free KV early
- Reorder groups by decode/prefill ratio

BatchLLM



- Accelerates large-batch LLM inference
- **Global Prefix Sharing:** Build prefix tree *ahead-of-time*; avoids LRU eviction; near-optimal KV reuse
- **Group Scheduling:** Execute requests as prefix-sharing groups; compute prefix once; free KV early
- Reorder groups by decode/prefill ratio

BatchLLM



- Accelerates large-batch LLM inference
- **Global Prefix Sharing:** Build prefix tree *ahead-of-time*; avoids LRU eviction; near-optimal KV reuse
- **Group Scheduling:** Execute requests as prefix-sharing groups; compute prefix once; free KV early
- Reorder groups by decode/prefill ratio

BatchLLM



- Accelerates large-batch LLM inference
- **Global Prefix Sharing:** Build prefix tree *ahead-of-time*; avoids LRU eviction; near-optimal KV reuse
- **Group Scheduling:** Execute requests as prefix-sharing groups; compute prefix once; free KV early
- Reorder groups by decode/prefill ratio

Workload Setup

- **Goal:** Evaluate inference performance of LLMs under short queries with large shared context.
- **Models:** Qwen 0.5B, Tiny Llama 1.1B, Llama 3.1 8B, served using vLLM/SGLang/Transformers.
- **Hardware:** NVIDIA RTX6000 48GB GPU.
- **Metrics:** Throughput (req/s), latency, GPU utilization.

Parameter	Value/Range
System Prompt Length	≈1500 tokens (shared across requests)
Query Rate	5–700 req/sec
Query Length	10–20 tokens
Output Length	30–50 tokens
Batch Size	Adaptive

Table: Workload Characteristics

Workload Setup



- **Goal:** Evaluate inference performance of LLMs under short queries with large shared context.
- **Models:** Qwen 0.5B, Tiny Llama 1.1B, Llama 3.1 8B, served using vLLM/SGLang/Transformers.
- **Hardware:** NVIDIA RTX6000 48GB GPU.
- **Metrics:** Throughput (req/s), latency, GPU utilization.

Parameter	Value/Range
System Prompt Length	≈1500 tokens (shared across requests)
Query Rate	5–700 req/sec
Query Length	10–20 tokens
Output Length	30–50 tokens
Batch Size	Adaptive

Table: Workload Characteristics

Workload Setup

- **Goal:** Evaluate inference performance of LLMs under short queries with large shared context.
- **Models:** Qwen 0.5B, Tiny Llama 1.1B, Llama 3.1 8B, served using vLLM/SGLang/Transformers.
- **Hardware:** NVIDIA RTX6000 48GB GPU.
- **Metrics:** Throughput (req/s), latency, GPU utilization.

Parameter	Value/Range
System Prompt Length	≈1500 tokens (shared across requests)
Query Rate	5–700 req/sec
Query Length	10–20 tokens
Output Length	30–50 tokens
Batch Size	Adaptive

Table: Workload Characteristics

Workload Setup



- **Goal:** Evaluate inference performance of LLMs under short queries with large shared context.
- **Models:** Qwen 0.5B, Tiny Llama 1.1B, Llama 3.1 8B, served using vLLM/SGLang/Transformers.
- **Hardware:** NVIDIA RTX6000 48GB GPU.
- **Metrics:** Throughput (req/s), latency, GPU utilization.

Parameter	Value/Range
System Prompt Length	≈1500 tokens (shared across requests)
Query Rate	5–700 req/sec
Query Length	10–20 tokens
Output Length	30–50 tokens
Batch Size	Adaptive

Table: Workload Characteristics

Experiment Setup: Engine Comparison



- Compare inference engines: **Transformers**, **vLLM**, **SGLang**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Experiment Setup: Engine Comparison



- Compare inference engines: **Transformers**, **vLLM**, **SGLang**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Experiment Setup: Engine Comparison



- Compare inference engines: **Transformers**, **vLLM**, **SGLang**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Experiment Setup: Engine Comparison



- Compare inference engines: **Transformers**, **vLLM**, **SGLang**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Experiment Setup: Engine Comparison



- Compare inference engines: **Transformers**, **vLLM**, **SGLang**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Batch of requests through various Engines

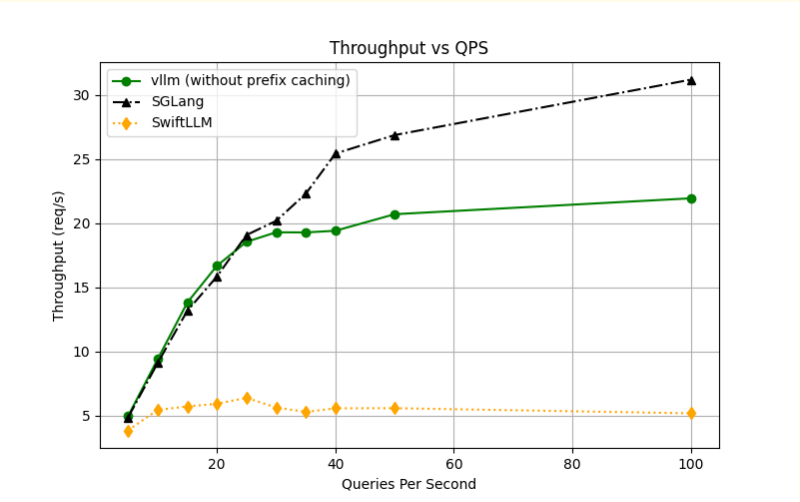


Figure: Throughput comparison across varying workloads

- **SwiftLLM:** lightweight vLLM clone (∼2000 LOC), but doesn't perform that well.
- **SGLang:** achieves best performance via radix-tree prefix sharing.

Technique Used	Time (s)
Transformers (batch = 1)	30.21
Transformers (batch = 2)	19.51
Transformers (batch = 3)	18.64
vLLM	0.45
SGLang (fork)	1.03
SGLang (run_batch)	0.40

Table: Batch of 24 requests

Experiment Setup: Request Timelines



- Aim: Measure time spent in **prefill** vs **decode** stages for 20 req/sec workload (sending a new request after every 0.05 seconds).
- Using vLLM V0 engine without chunked prefill.
- Tool: Custom logging of per-request timestamps.

Figure: Simplified per-request timeline

Experiment Setup: Request Timelines



- Aim: Measure time spent in **prefill** vs **decode** stages for 20 req/sec workload (sending a new request after every 0.05 seconds).
- Using vLLM V0 engine without chunked prefill.
- Tool: Custom logging of per-request timestamps.

Figure: Simplified per-request timeline

Experiment Setup: Request Timelines



- Aim: Measure time spent in **prefill** vs **decode** stages for 20 req/sec workload (sending a new request after every 0.05 seconds).
- Using vLLM V0 engine without chunked prefill.
- Tool: Custom logging of per-request timestamps.

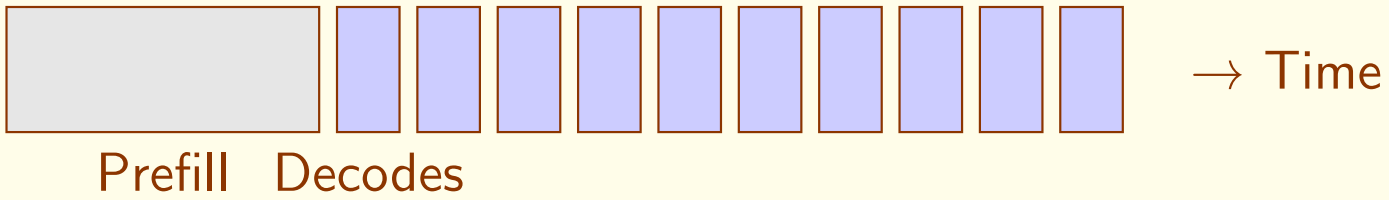


Figure: Simplified per-request timeline

Results: LLM Request Timelines

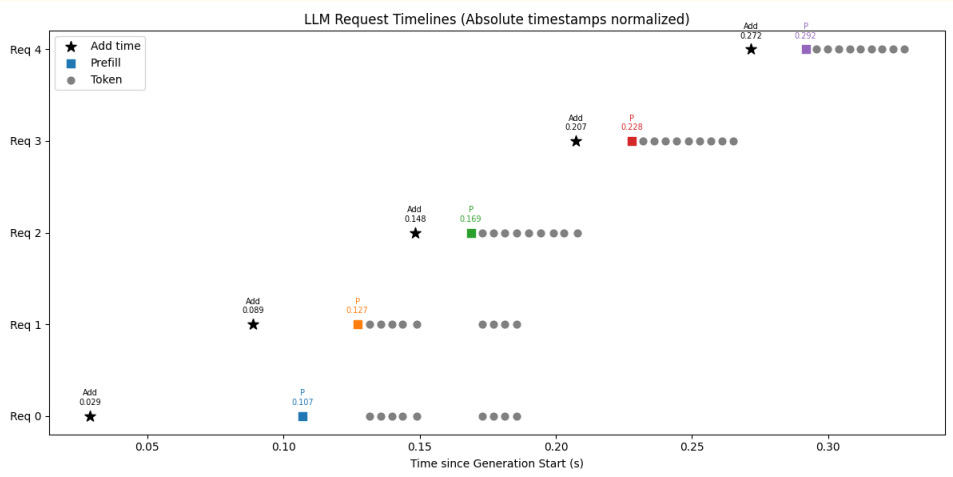


Figure: 20 Requests/sec – No chunked prefill – vLLM V0

- Prefill dominates runtime due to large prompt.
- Prefill and decode cannot overlap here.
- Low TTFT but high total latency.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: LLM Request Timelines

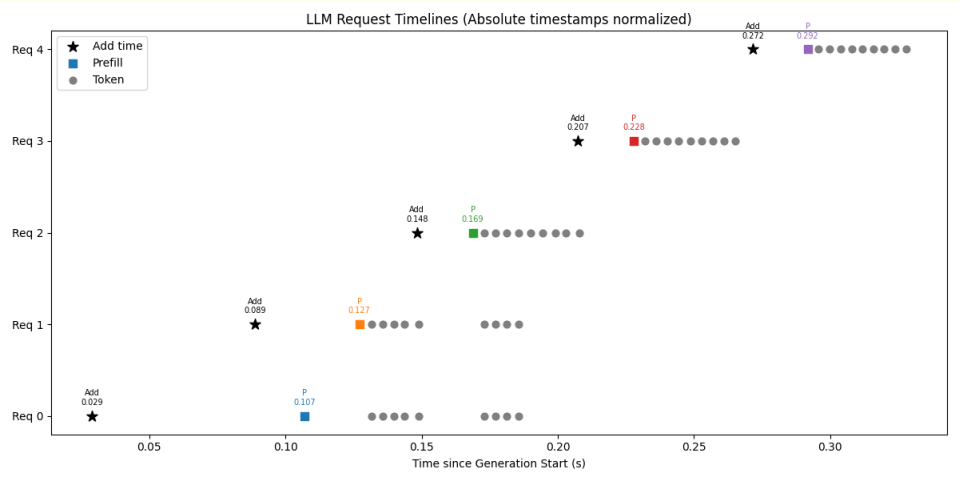


Figure: 20 Requests/sec – No chunked prefill – vLLM V0

- Prefill dominates runtime due to large prompt.
- Prefill and decode cannot overlap here.
- Low TTFT but high total latency.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: LLM Request Timelines

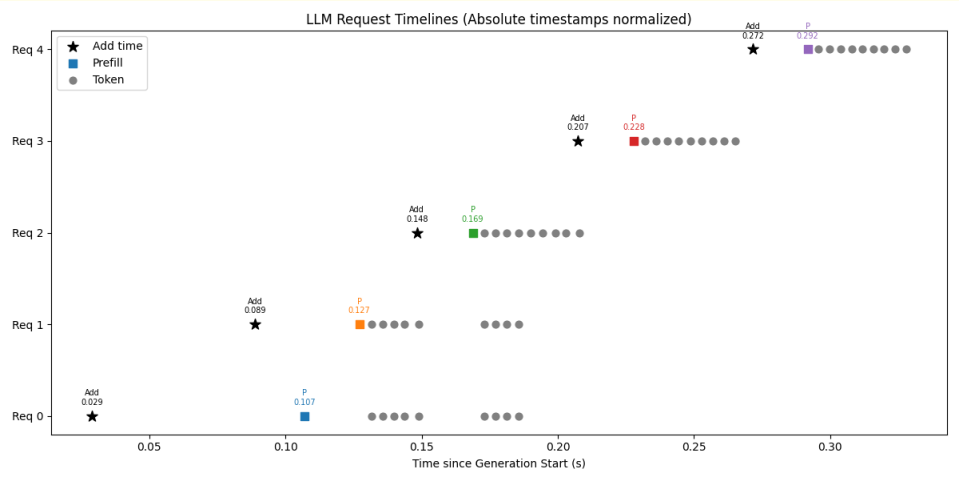


Figure: 20 Requests/sec – No chunked prefill – vLLM V0

- Prefill dominates runtime due to large prompt.
- Prefill and decode cannot overlap here.
- Low TTFT but high total latency.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: Throughput vs Load for vLLM

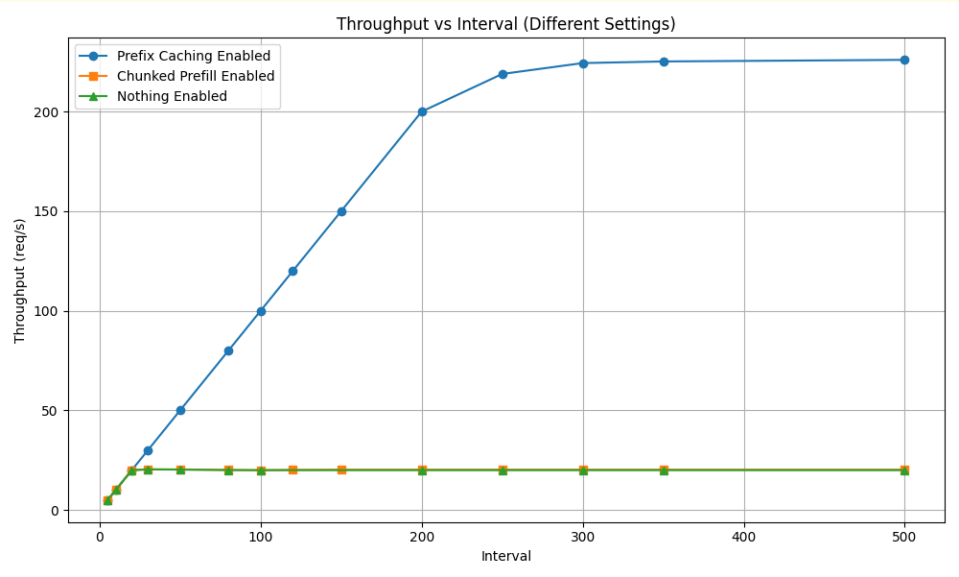


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: Throughput vs Load for vLLM

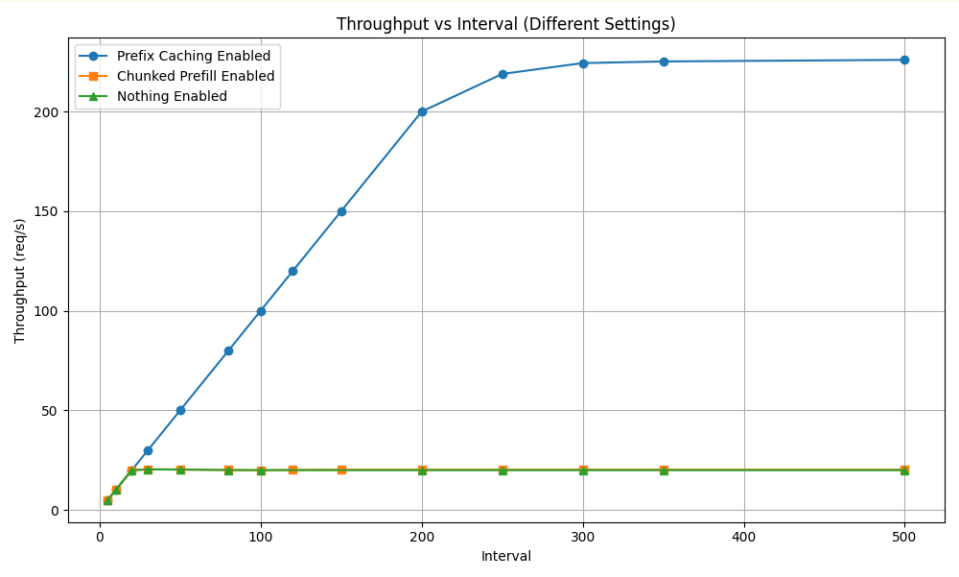


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: Throughput vs Load for vLLM

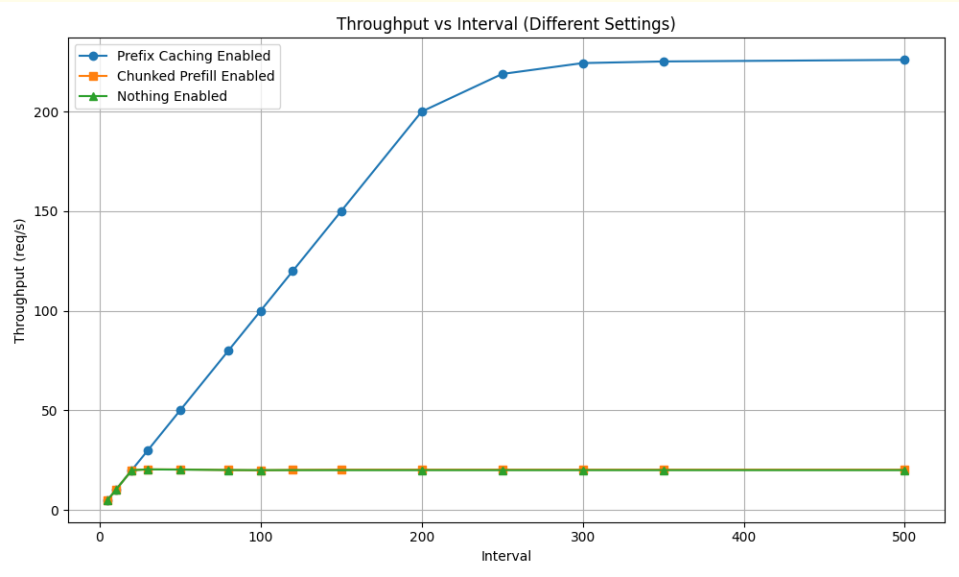


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: Throughput vs Load for vLLM

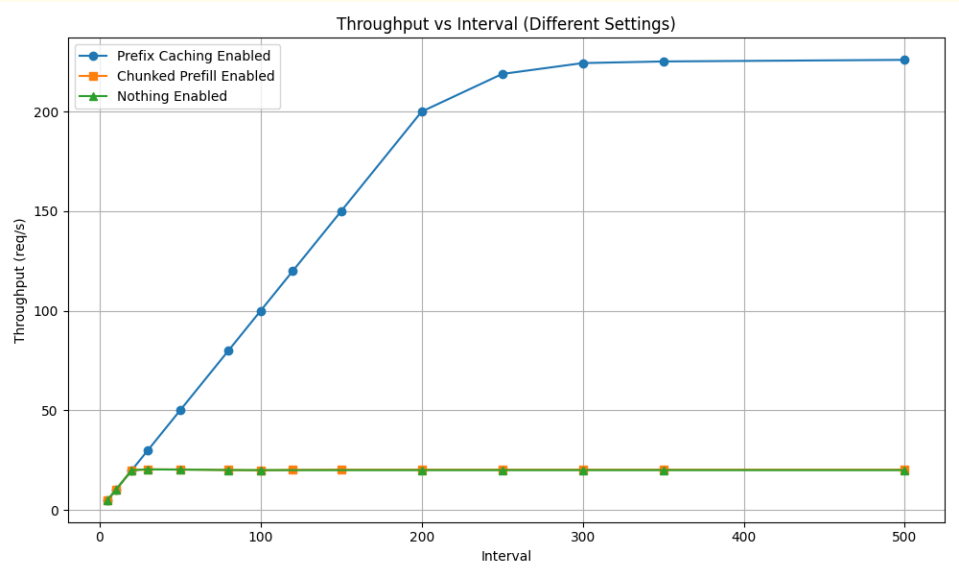


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: Throughput vs Load for vLLM

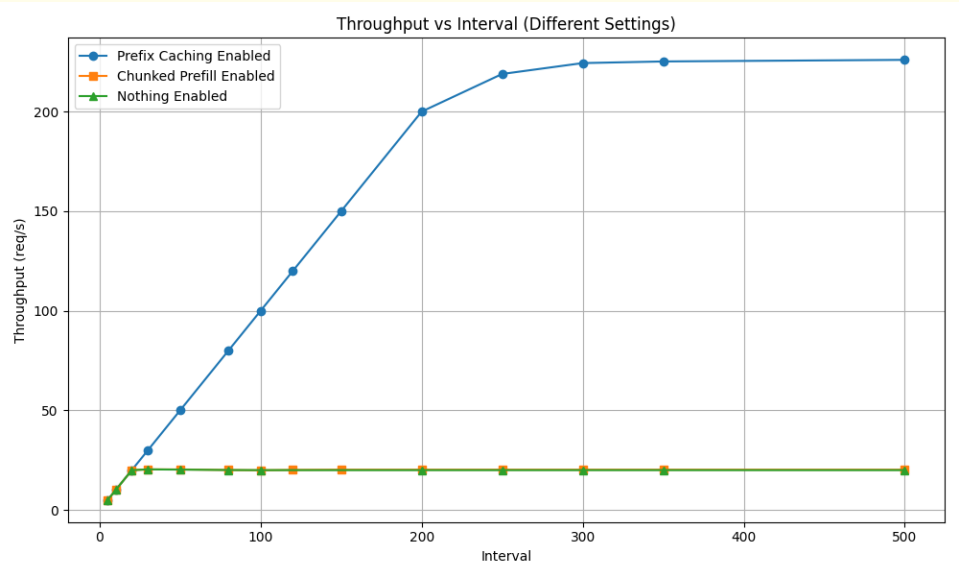


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Key GPU Metrics



- Tools: **Nsight Systems (NSYS)** and **DCGMI**
- **SM Activity:** % time with at least one active warp
- **SM Occupancy:** resident warps / max warps
- **Tensor Core Utilization:** % cycles tensor pipe active

Key GPU Metrics



- Tools: **Nsight Systems (NSYS)** and **DCGMI**
- **SM Activity:** % time with at least one active warp
- **SM Occupancy:** resident warps / max warps
- **Tensor Core Utilization:** % cycles tensor pipe active

Key GPU Metrics



- Tools: **Nsight Systems (NSYS)** and **DCGMI**
- **SM Activity:** % time with at least one active warp
- **SM Occupancy:** resident warps / max warps
- **Tensor Core Utilization:** % cycles tensor pipe active

Key GPU Metrics



- Tools: **Nsight Systems (NSYS)** and **DCGMI**
- **SM Activity:** % time with at least one active warp
- **SM Occupancy:** resident warps / max warps
- **Tensor Core Utilization:** % cycles tensor pipe active

NSYS Run on Matrix Multiplication

- Various Phases: Host to device copy, kernel computation, and device to host copy.
- Compute metrics reach high values during kernel execution phase.

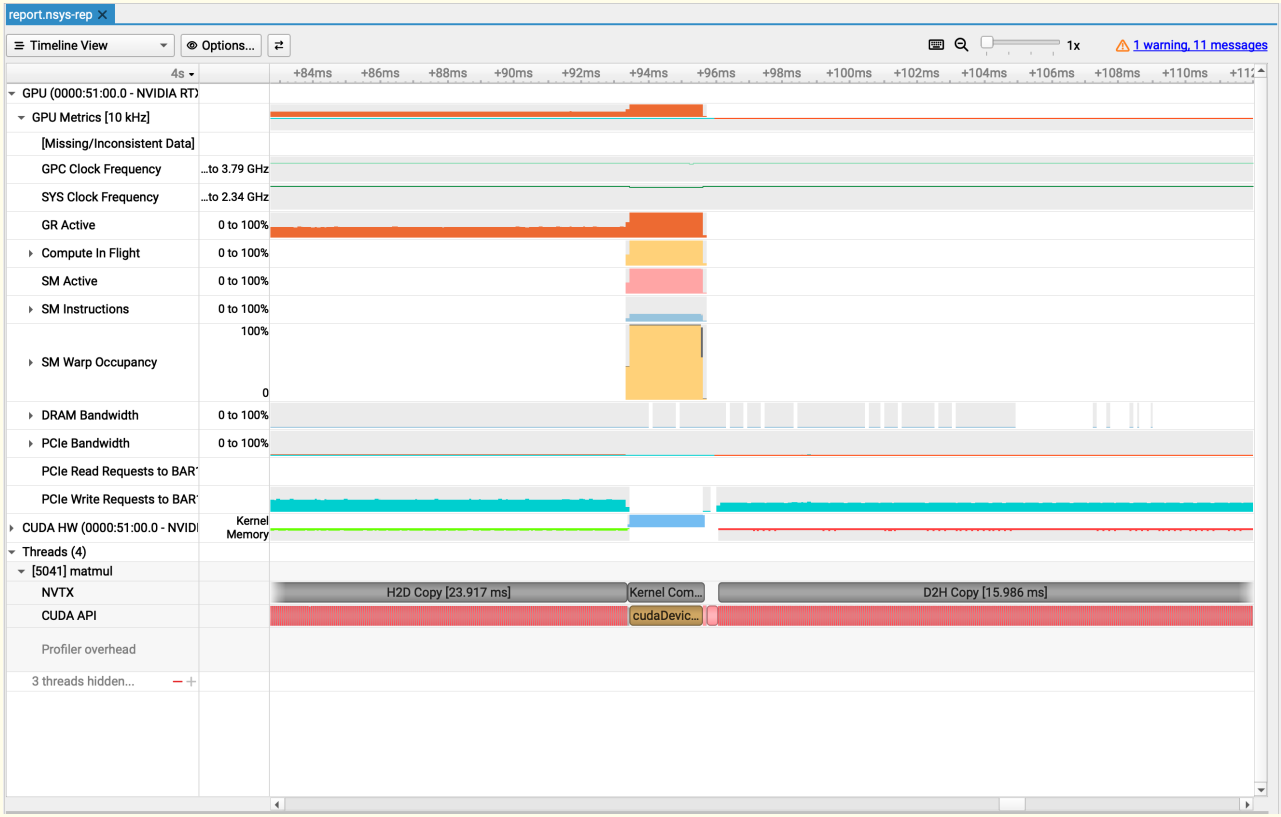


Figure: NSYS Metrics Snapshot

Multiple Families - Generation of such a workload

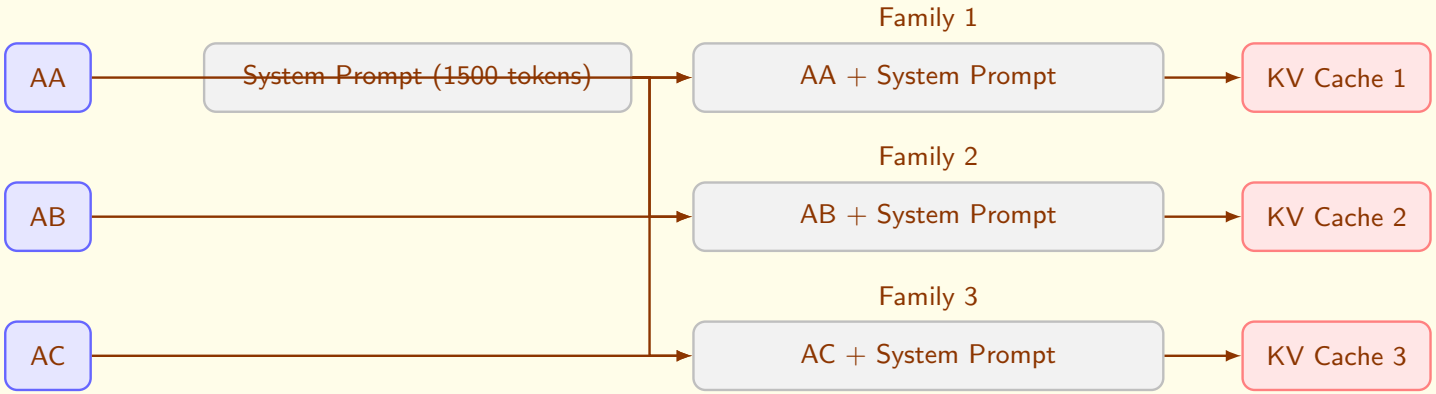


Figure: Multiple Families' Workload Generation

- What we have already - a large system prompt of around 1500 tokens.
- A Naive Way - Write down multiple such system prompts, and 100 queries based on each one of them.
- A Better Way - Add unique identifiers ("AA", "AB") to the existing system prompt.
- Different block hashes → separate KV Cache blocks for each family.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Multiple Families - Generation of such a workload

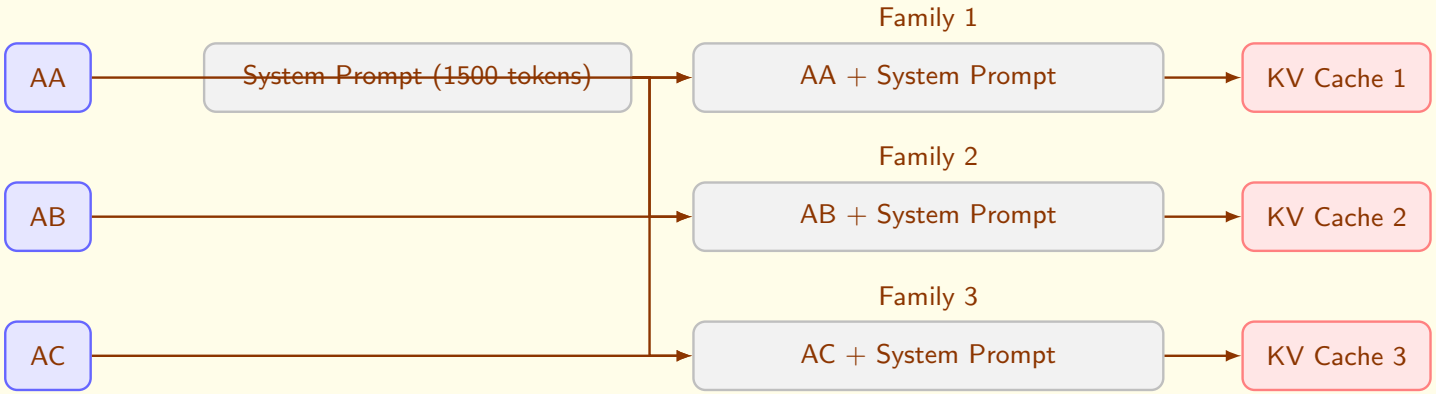


Figure: Multiple Families' Workload Generation

- What we have already - a large system prompt of around 1500 tokens.
- A Naive Way - Write down multiple such system prompts, and 100 queries based on each one of them.
- A Better Way - Add unique identifiers ("AA", "AB") to the existing system prompt.
- Different block hashes → separate KV Cache blocks for each family.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Multiple Families - Generation of such a workload

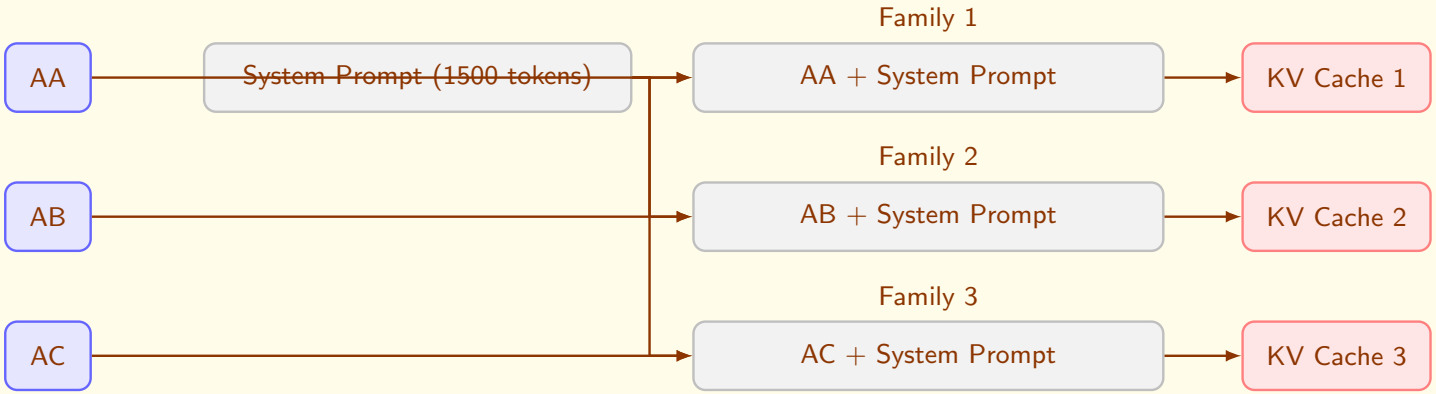


Figure: Multiple Families' Workload Generation

- What we have already - a large system prompt of around 1500 tokens.
- A Naive Way - Write down multiple such system prompts, and 100 queries based on each one of them.
- A Better Way - Add unique identifiers ("AA", "AB") to the existing system prompt.
- Different block hashes → separate KV Cache blocks for each family.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Multiple Families - Generation of such a workload

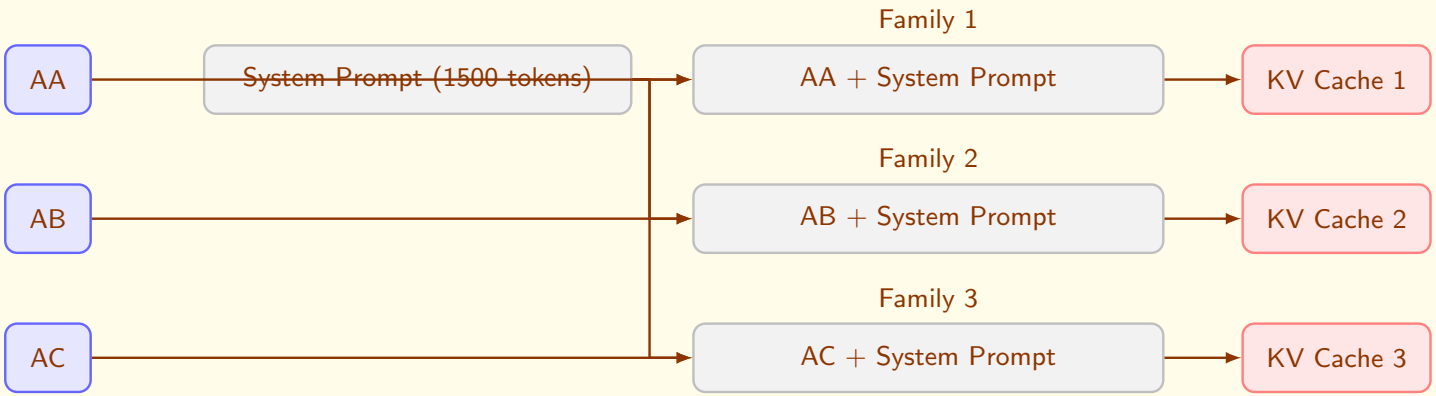


Figure: Multiple Families' Workload Generation

- What we have already - a large system prompt of around 1500 tokens.
- A Naive Way - Write down multiple such system prompts, and 100 queries based on each one of them.
- A Better Way - Add unique identifiers ("AA", "AB") to the existing system prompt.
- Different block hashes $\rightarrow$ separate KV Cache blocks for each family.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Effect of different Models

Model Chosen	10 families, each with 1000 queries	1 family with 10000 requests
Qwen 0.5B	294.228 req/s	308.750 req/s
Llama 7B	43.923 req/s	124.347 req/s
LongChat 13B	22.415 req/s	70.097 req/s

Table: Throughput with multiple families across various models

- As the model size increases, the throughput degradation becomes more pronounced.
- Primarily because of - (1) Increased attention computation overhead (2) Increase in the working set size - more transactions with global memory.

Effect of different Models



Model Chosen	10 families, each with 1000 queries	1 family with 10000 requests
Qwen 0.5B	294.228 req/s	308.750 req/s
Llama 7B	43.923 req/s	124.347 req/s
LongChat 13B	22.415 req/s	70.097 req/s

Table: Throughput with multiple families across various models

- As the model size increases, the throughput degradation becomes more pronounced.
- Primarily because of - (1) Increased attention computation overhead (2) Increase in the working set size - more transactions with global memory.

Same Family → Single Batch



Number of Families	Number of queries in each family	Max Batch Size	Ordered by Family ID	Random Permutation
10	10	10	12.28	11.981
10	100	100	59.601	55.704
10	100	200	61.693	60.923
100	100	100	42.882	39.172

Table: Comparison of Throughput (req/sec) when merging queries from the same family into a single batch vs not merging

- Merging same-family queries into one batch boosts throughput
- Reason: L2 cache is too small to hold KV caches of many families → more HBM loads
- Grouping same-family queries reduces unique KV blocks per batch

Same Family → Single Batch

Number of Families	Number of queries in each family	Max Batch Size	Ordered by Family ID	Random Permutation
10	10	10	12.28	11.981
10	100	100	59.601	55.704
10	100	200	61.693	60.923
100	100	100	42.882	39.172

Table: Comparison of Throughput (req/sec) when merging queries from the same family into a single batch vs not merging

- Merging same-family queries into one batch boosts throughput
- Reason: L2 cache is too small to hold KV caches of many families → more HBM loads
- Grouping same-family queries reduces unique KV blocks per batch

Same Family → Single Batch



Number of Families	Number of queries in each family	Max Batch Size	Ordered by Family ID	Random Permutation
10	10	10	12.28	11.981
10	100	100	59.601	55.704
10	100	200	61.693	60.923
100	100	100	42.882	39.172

Table: Comparison of Throughput (req/sec) when merging queries from the same family into a single batch vs not merging

- Merging same-family queries into one batch boosts throughput
- Reason: L2 cache is too small to hold KV caches of many families → more HBM loads
- Grouping same-family queries reduces unique KV blocks per batch

Prior Prompt Prefilling

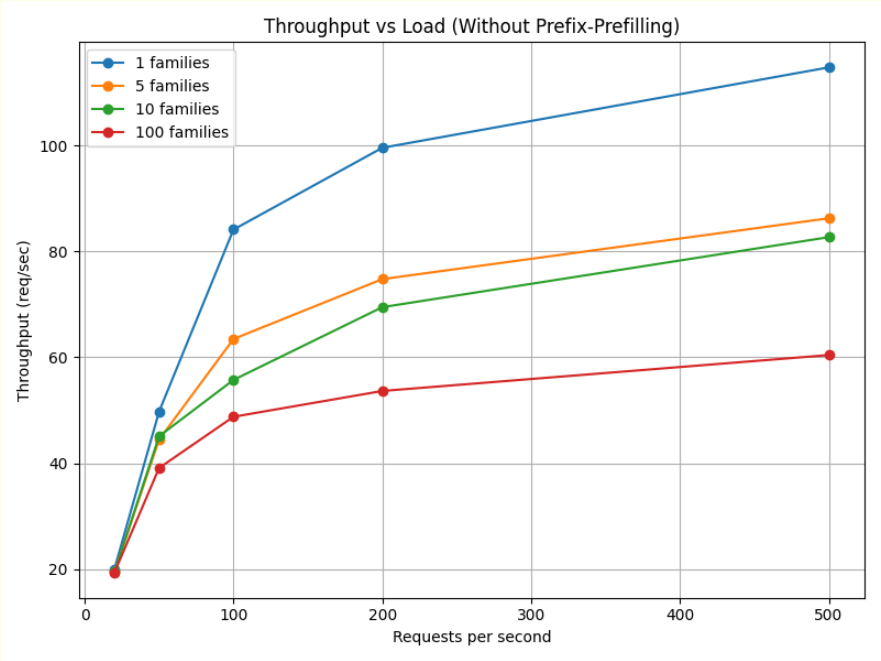


Figure: Without prefix pre-filling

Prior Prompt Prefilling

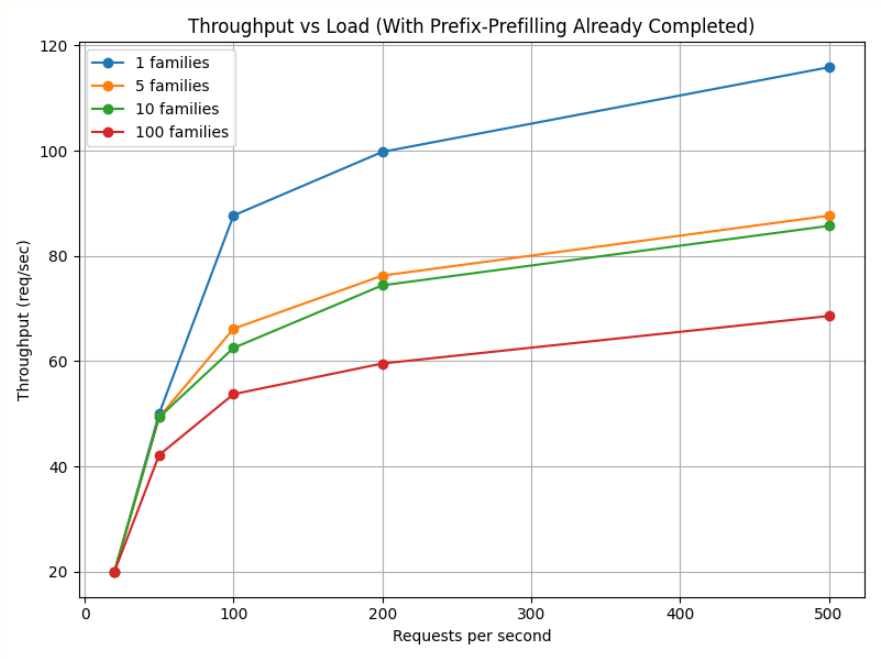


Figure: With prefix pre-filling

- System prompts prefilled → KV already in HBM
- Still large throughput differences across families

Prior Prompt Prefilling

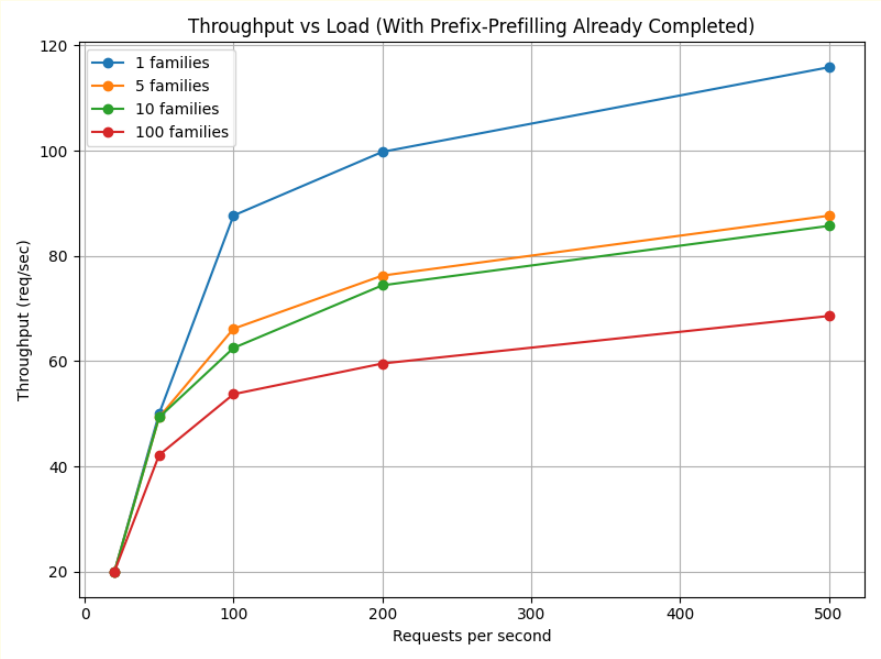


Figure: With prefix pre-filling

- System prompts prefilled → KV already in HBM
- Still large throughput differences across families

Prior Prompt Prefilling

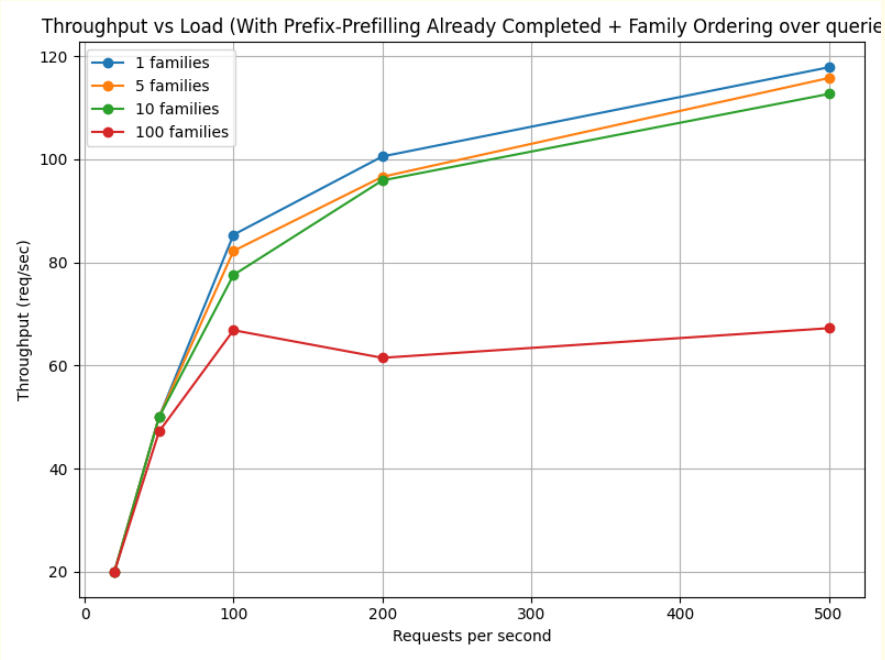


Figure: Ordered by family IDs

- Main bottleneck: memory bandwidth to load KV blocks from HBM to L2
- Grouping / ordering families reduces unique KV blocks → higher throughput

Prior Prompt Prefilling



Figure: Ordered by family IDs

- Main bottleneck: memory bandwidth to load KV blocks from HBM to L2
- Grouping / ordering families reduces unique KV blocks → higher throughput

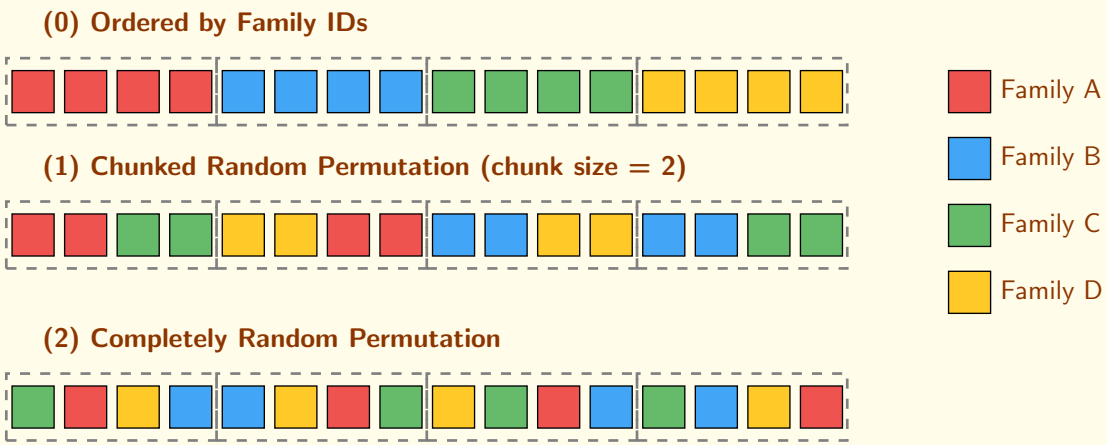
Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

What all parameters we can control?

- Number of Families
- Workload - requests per second
- Max Batch Size
- The nature of the batch - whether ordered by family IDs or random permutation

Batch Nature Levels

- 0: Ordered by family IDs
- 1: We make chunks of size 50 from the sorted list and permute each chunk randomly (allows us to have some level of prefix sharing between consecutive queries)
- 2: Completely random permutation



Varying Everything Together

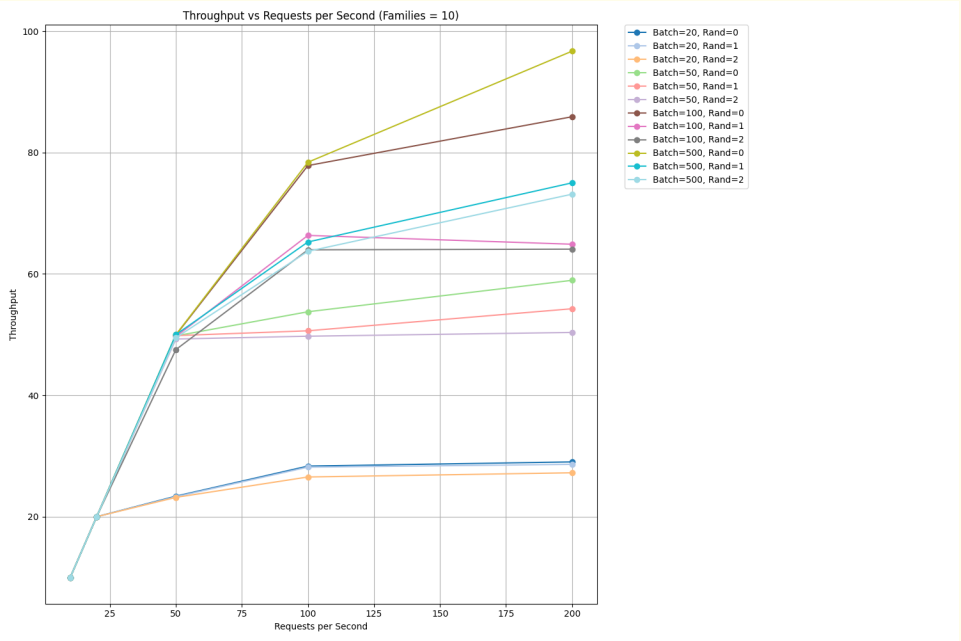


Figure: Throughput vs Workload by varying other parameters (10 families)

- Increase in batch size $\rightarrow$ higher throughput.
- A batch nature level of 1 is roughly similar to level 2 $\rightarrow$ even a small amount of mixing leads to significant degradation.
- A homogeneous batch of smaller size at times performs siginificantly better than a larger heterogeneous batch.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Varying Everything Together

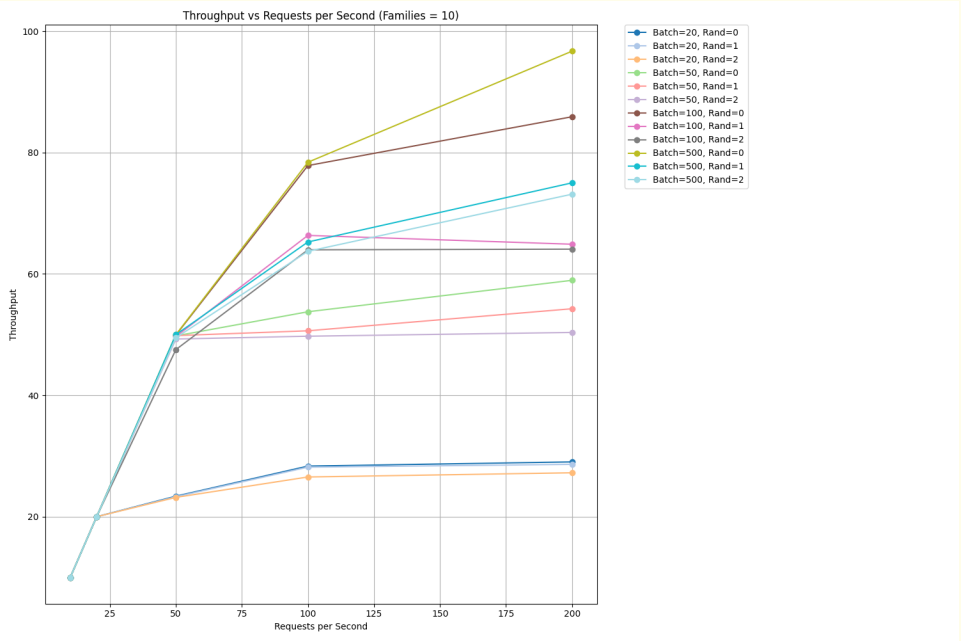


Figure: Throughput vs Workload by varying other parameters (10 families)

- Increase in batch size $\rightarrow$ higher throughput.
- A batch nature level of 1 is roughly similar to level 2 $\rightarrow$ even a small amount of mixing leads to significant degradation.
- A homogeneous batch of smaller size at times performs siginificantly better than a larger heterogeneous batch.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Varying Everything Together

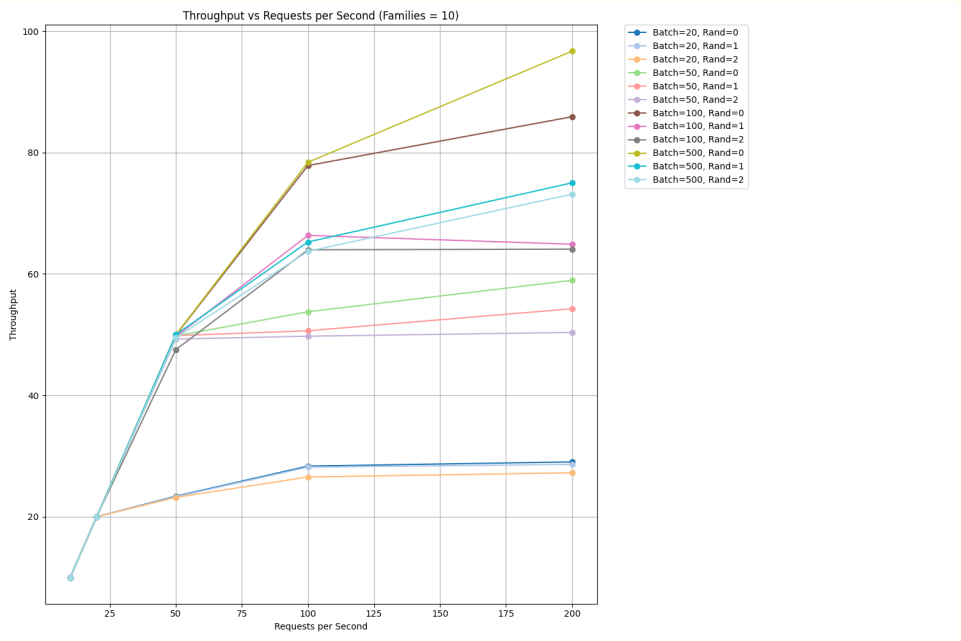


Figure: Throughput vs Workload by varying other parameters (10 families)

- Increase in batch size $\rightarrow$ higher throughput.
- A batch nature level of 1 is roughly similar to level 2 $\rightarrow$ even a small amount of mixing leads to significant degradation.
- A homogeneous batch of smaller size at times performs siginificantly better than a larger heterogeneous batch.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Varying Chunk Size while Permuting

Table: Throughput across different k levels

k	**Throughput (req/sec)**
200	89.35
300	87.49
400	86.50
499	88.19
500	112.66
1000	112.41

- Before passing the requests to the model, we make chunks of size k, and permute each chunk randomly.
- A k value of 499 essentially means that in the first batch we have 499 requests of one family and 1 request of another family
- However the throughput is much lower than a k value of 500.
- Even one request of a different family will cause L2 cache thrashing.

Varying Chunk Size while Permuting

Table: Throughput across different k levels

k	**Throughput (req/sec)**
200	89.35
300	87.49
400	86.50
499	88.19
500	112.66
1000	112.41

- Before passing the requests to the model, we make chunks of size k, and permute each chunk randomly.
- A k value of 499 essentially means that in the first batch we have 499 requests of one family and 1 request of another family
- However the throughput is much lower than a k value of 500.
- Even one request of a different family will cause L2 cache thrashing.

Varying Chunk Size while Permuting

Table: Throughput across different k levels

k	**Throughput (req/sec)**
200	89.35
300	87.49
400	86.50
499	88.19
500	112.66
1000	112.41

- Before passing the requests to the model, we make chunks of size k, and permute each chunk randomly.
- A k value of 499 essentially means that in the first batch we have 499 requests of one family and 1 request of another family
- However the throughput is much lower than a k value of 500.
- Even one request of a different family will cause L2 cache thrashing.

Varying Chunk Size while Permuting

Table: Throughput across different k levels

k	**Throughput (req/sec)**
200	89.35
300	87.49
400	86.50
499	88.19
500	112.66
1000	112.41

- Before passing the requests to the model, we make chunks of size k, and permute each chunk randomly.
- A k value of 499 essentially means that in the first batch we have 499 requests of one family and 1 request of another family
- However the throughput is much lower than a k value of 500.
- Even one request of a different family will cause L2 cache thrashing.

DRAM Bandwidth Utilization - for Multiple Families

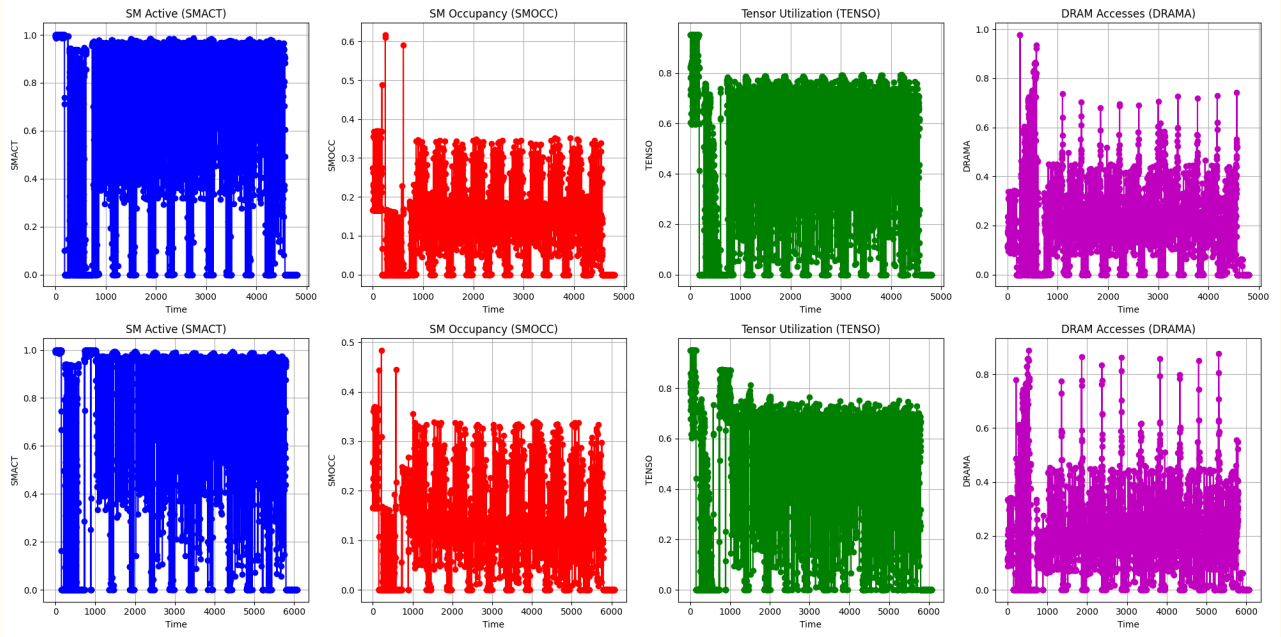


Figure: Comparison for $k = 500$ and $k = 499$

- L2 cache hit/miss rates $\rightarrow$ difficult to capture, as there are thousands of kernels forked.
- DRAMA - fraction of cycles where data was sent to or received from device memory.
- Average DRAMA for $k = 500 \rightarrow 0.18$; Average DRAMA for $k = 499 \rightarrow 0.22$.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

DRAM Bandwidth Utilization - for Multiple Families

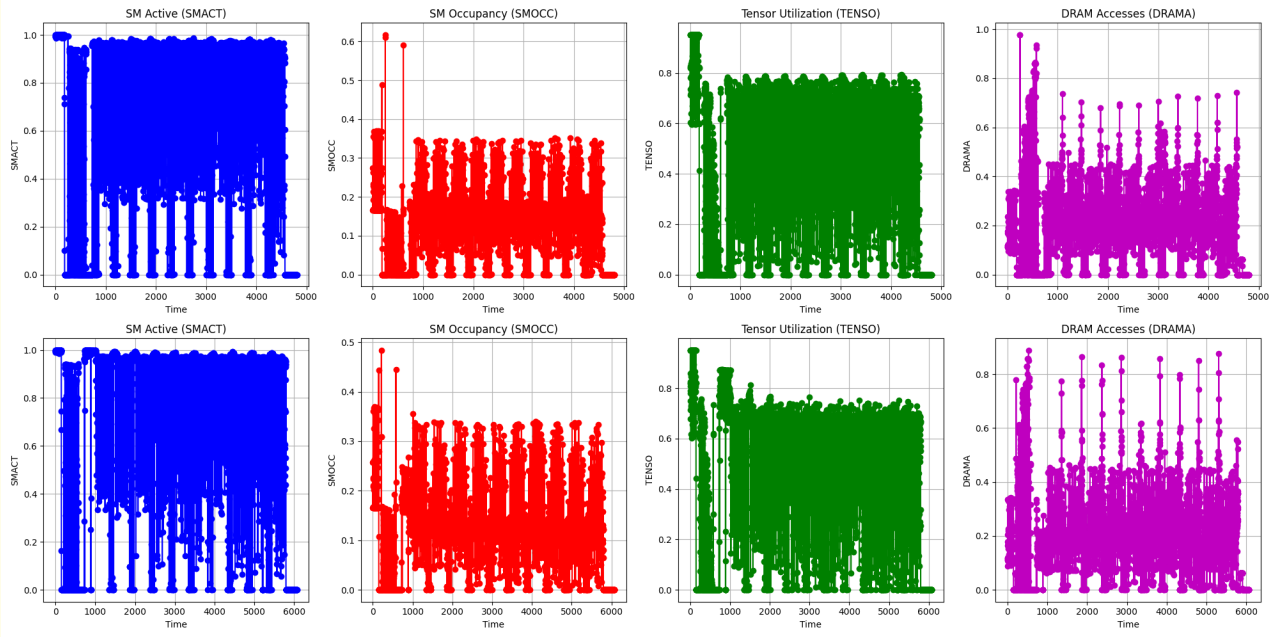


Figure: Comparison for $k = 500$ and $k = 499$

- L2 cache hit/miss rates $\rightarrow$ difficult to capture, as there are thousands of kernels forked.
- DRAMA - fraction of cycles where data was sent to or received from device memory.
- Average DRAMA for $k = 500 \rightarrow 0.18$; Average DRAMA for $k = 499 \rightarrow 0.22$.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

DRAM Bandwidth Utilization - for Multiple Families

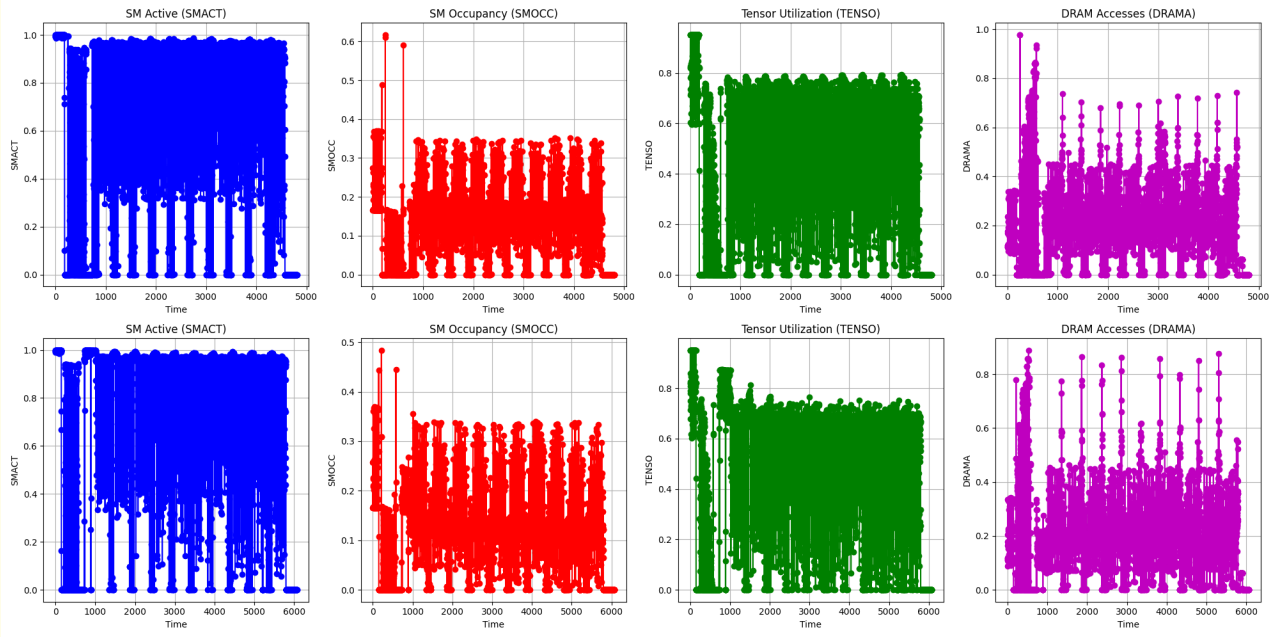


Figure: Comparison for $k = 500$ and $k = 499$

- L2 cache hit/miss rates $\rightarrow$ difficult to capture, as there are thousands of kernels forked.
- DRAMA - fraction of cycles where data was sent to or received from device memory.
- Average DRAMA for $k = 500 \rightarrow 0.18$; Average DRAMA for $k = 499 \rightarrow 0.22$.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Future Steps - Improved Scheduler



- Incorporate L2 cache size and batch working-set size
- Track number and size of KV blocks accessed
- Dynamically decide between:
  - smaller, homogeneous batches
  - larger, mixed batches

Future Steps - SM-Level Optimization



- Goal: increase per-SM L1 cache hit rate
- Schedule same-family requests on nearby / same SMs
- Assign kernels IDs derived from family IDs
- Prefer SMs already serving that family

Future Steps - KV Cache Offloading



- HBM may become insufficient as families grow
- Need an efficient KV eviction/offload strategy
- Eviction guided by prefix-sharing intensity

Future Steps - Response Quality



- Improve response quality for shared-context workloads
- Fine-tune on shared contexts
- Compare against baseline using system-prompt injection



Thank You